

多媒體技術與應用

第 1 組期末專案報告

JetBot 道路跟隨與交通路牌辨識整合系統

以最新版 `Final.ipynb` 為核心之雙模型自主駕駛控制

組別：第 1 組

班級、姓名與學號：

電資二 113820020 林政德

電資二 113820033 謝奕宏

電資二 112820034 呂伊茹

指導教授：陳彥霖（Yen-Lin Chen）, Ph.D.

學校：國立臺北科技大學 電資學士班

日期：2026.06.20

分工摘要：

林政德：JetBot 硬體操作、賽道測試、道路循跡參數調整與實體 Demo 協作（33%）

謝奕宏：雙模型整合、`Final.ipynb` 控制邏輯、模型部署包、測試與除錯（34%）

呂伊茹：路牌資料整理、實驗紀錄、報告內容與成果彙整（33%）

1. 實驗內容：

本次期末專案的目標，是整合本學期 Project 5 的道路跟隨技術與 Project 6 的交通路牌辨識技術，在 NVIDIA Jetson Nano JetBot 自走車上建構一套完整的道路場景智能控制系統。系統以最新版 `Final.ipynb` 為唯一主控程式，使用單一 CSI 相機取得即時影像，同時執行道路中心點預測與交通路牌辨識，再依辨識結果控制左右馬達，使 JetBot 能完成賽道並執行指定交通動作。

依照期末專案規定，系統必須完成下列功能：

- JetBot 能自主完成道路賽道，並盡量避免輪胎壓線或超出車道。
- 偵測到 pedestrian（當心行人）時，降低速度後繼續行駛。
- 偵測到 blocked（道路封閉）時，在路牌前停止，且停止位置不得超過路牌。
- 偵測到 stop（停車再開）時，原地停止 2 秒後再繼續行駛。
- 偵測到 rail（鐵路平交道）時，原地停止 5 秒後再繼續行駛。
- 遇到遠處、背景或賽道旁的假路牌時，不應錯誤執行動作。

系統使用的模型

子系統	模型與任務	輸入	輸出	JetBot 部署方式
道路跟隨	ResNet-18 座標迴歸	224×224 RGB	正規化道路目標座標 (X, Y)	ONNX 轉 TensorRT，使用 TRTModule 載入
路牌辨識	YOLOv4-tiny 物件偵測	416×416 BGR/RGB 影像	邊界框、信心度、類別 ID	YOLO 權重轉 ONNX，再轉 TensorRT

道路模型以重新收集的 800 張 224×224 賽道影像訓練。影像檔名使用 `xy_{X}_{Y}_{uuid}.jpg` 格式保存人工標定的目標座標，模型最後一層修改為 `Linear(512, 2)`，輸出道路前進方向所需的 X、Y 座標。

路牌模型以 151 張人工標註影像訓練，辨識類別與控制動作如下：

類別 ID	類別	路牌	最新版 Final.ipynb 動作
0	stop		連續確認後原地停止 2 秒，設定 10 秒冷卻時間，再等待新影格恢復行駛
1	rail		連續確認後原地停止 5 秒，設定 10 秒冷卻時間，再等待新影格恢復行駛
2	pedestrian		將巡航速度調整為 0.7 倍，最後一次確認後持續減速 1 秒
3	blocked		立即停止並設定永久鎖定，重新執行啟動單元前不得恢復行駛

最新版 Final.ipynb 的程式結構

Final.ipynb 分為三個可獨立執行的測試單元：

1. 單元一：純道路跟隨測試

使用 224×224 相機影像與道路 TensorRT 模型，提供 Speed、P Gain、D Gain、Bias 滑桿，用於現場調整循跡參數。

2. 單元二：純交通路牌測試

使用 416×416 相機影像與 YOLOv4-tiny TensorRT 模型，單獨測試四種路牌的停車、減速、冷卻及永久停止邏輯。

3. 單元三：最終整合模式

使用單一 416×416 相機影像，原始影像送入 YOLOv4-tiny，並將同一影格縮放至 224×224 後送入 ResNet-18，使兩套模型共用相機資源並共同控制 JetBot。

Camera 416x416

└ YOLOv4-tiny → 信心度／框寬／連續影格確認 → 路牌動作狀態

└ Resize 224x224 → ResNet-18 → 道路座標 → PD 控制 → 左右馬達

最新版控制安全機制

為配合只有三次實體測試機會的期末驗收，最新版程式加入下列穩定性與安全機制：

- YOLO 推論信心度門檻設為 0.60。
- 每一類路牌使用獨立框寬門檻：Stop 50 px、Rail 30 px、Pedestrian 50 px、Blocked 50 px。
- 路牌必須連續 2 個影格符合信心度及框寬條件後才會觸發。
- 多個路牌同時成立時，以 blocked > stop > rail > pedestrian 的安全優先順序決策。
- Stop 與 Rail 停止結束後直接結束該次 callback，等待相機的新影格，不使用停止前的舊影像重新起步。
- Pedestrian 使用 1 秒保持時間，避免單幀漏檢造成速度在 1.0 倍與 0.7 倍之間快速跳動。
- Blocked 使用永久鎖定狀態，防止 callback 或操作流程意外讓車輛再次前進。
- callback 發生任何模型、影像或控制例外時，立即停止馬達並進入安全鎖定。
- 道路模型輸出會先檢查長度、NaN 與 Inf，再限制於 $[-1, 1]$ 範圍。
- 彎道速度依轉向量自動降低，最低保留設定速度的 60%，減少高速進入急彎造成壓線或出界的機率。

2. 實驗過程及結果：

實驗過程

步驟一：道路資料收集與 ResNet-18 訓練

使用 JetBot 的 CSI 相機擷取 224×224 道路影像，並透過 Gamepad 或影像介面標記車輛應前往的道路目標點。最終整理出 800 張影像，座標範圍皆位於有效影像範圍內。訓練時將座標依下式正規化：

$$x_{norm} = \frac{x - 112}{112}, \quad y_{norm} = \frac{y - 112}{112}$$

訓練資料使用亮度、對比度、飽和度、色相、隨機仿射與水平翻轉等資料增強。水平翻轉時同步將 X 標籤取負值，避免影像與座標標籤不一致。模型採用 ResNet-18，將原本分類用的最後一層改為兩個連續值輸出，並以 MSE Loss 與 Adam 最佳化器進行訓練。

道路控制使用 PD 控制器。為避免正規化 Y 座標為負值時造成角度反轉，最新版程式先計算正向距離：

$$y_{forward} = \max(1 - y, 0.05)$$

再計算：

$$angle = \text{atan2}(x, y_{forward})$$

$$steering = K_p \cdot angle + K_d \cdot (angle - angle_{last}) + bias$$

左右馬達輸出為：

$$left = \text{clamp}(base_speed + steering, 0, 1)$$

$$right = \text{clamp}(base_speed - steering, 0, 1)$$

圖 2：道路模型離線預測點與人工標記點比較。



道路預測視覺化

步驟二：路牌資料標註與 YOLOv4-tiny 訓練

路牌資料集包含 stop、rail、pedestrian、blocked 四種類別，共 151 張影像。使用 Labellmg 進行 YOLO 格式標註，並固定類別順序為：

```
0 = stop
1 = rail
2 = pedestrian
3 = blocked
```

資料以 8:1:1 比例切分為訓練集 121 張、驗證集 15 張及測試集 15 張，隨機種子設為 42。訓練時加入 HSV 色偏、模糊、亮度與對比度、水平翻轉及幾何平移等增強，並採用 CIoU 邊界框損失、正樣本信心度加權、Warmup、Cosine Annealing 與最佳權重追蹤。

1000 Epochs 訓練歷程如下：

Epoch	總 Loss	Box Loss	Conf Loss	Class Loss	Validation F1
1	11.9264	6.4371	4.0796	1.4097	0.0020
100	1.3671	1.2846	0.0568	0.0257	0.6726
500	0.6097	0.5887	0.0187	0.0022	0.8787
1000	0.1974	0.1822	0.0146	0.0006	0.9330

圖 3：YOLOv4-tiny 隔離測試集 15 張影像的辨識結果。



YOLOv4-tiny 測試集結果

步驟三：模型轉換與 JetBot 部署

道路模型先由 PyTorch 權重匯出成 ONNX，再於 JetBot 使用 NVIDIA trtexec 編譯成 TensorRT 引擎，最後包裝為 `best_steering_model_xy_trt.pth` 供 TRTModule 載入。YOLOv4-tiny 則將 `.cfg` 與 `.weights` 透過 JetBot 專案既有的 `yolo_to_onnx.py`、`onnx_to_tensorrt.py` 轉換為 TensorRT 引擎。

最新版 `Final.ipynb` 已將兩套模型的首次轉換流程集中於 Cell 2。部署包內包含：

```
Final.ipynb
road_following_model/best_steering_model_xy.pth
road_following_model/best_steering_model_xy.onnx
yolo/yolov4-tiny-416.cfg
yolo/yolov4-tiny-416.weights
yolo/obj.names
```

將部署包內容複製至 JetBot 既有的 `~/trt_yolov4-tiny-master/` 後，先執行 Cell 2 產生兩套 TensorRT 引擎，重啟 Kernel 釋放轉換時使用的 GPU 記憶體，再依序執行單元三 C1 至 C6。

步驟四：雙模型影像分流與控制整合

單元三使用一個 416×416 相機實例。每次相機更新觸發 `execute_final(change)`，處理順序如下：

1. 先執行 YOLOv4-tiny 推論，取得框座標、類別與信心度。

2. 依四類獨立框寬、10 秒冷卻、連續兩幀確認及安全優先順序選擇動作。
3. 若是 Stop 或 Rail，停止指定秒數後立即返回，等待新影格。
4. 若是 Blocked，設定永久停止鎖定；若 callback 發生例外，設定安全停止鎖定。
5. 若可繼續行駛，將同一影格縮放至 224×224 ，執行道路模型推論。
6. 使用 PD 控制器計算轉向，並依轉向量套用彎道速度係數。
7. 將馬達輸出、道路預測點、路牌框、信心度、框寬及目前動作顯示於控制面板。

彎道自動降速公式為：

$$curve_scale = \max(0.60, 1 - 1.5 \times |steering|)$$

$$base_speed = Speed \times sign_multiplier \times curve_scale$$

其中 `sign_multiplier` 在 pedestrian 狀態為 0.7，其餘正常行駛狀態為 1.0。

預期實驗結果

- 道路模型可持續輸出有效的 X、Y 道路目標座標，JetBot 能依 PD 控制完成直線與彎道。
- 急彎時系統自動降低速度，降低輪胎壓線及衝出賽道的機率。
- Stop 連續確認後原地停止 2 秒，Rail 連續確認後原地停止 5 秒，且停止後使用新影格重新決策。
- Pedestrian 觸發後速度降為 0.7 倍，短暫漏檢時仍維持減速，通過後恢復巡航速度。
- Blocked 在路牌前停止並保持鎖定，不因後續影格或 callback 再次觸發而恢復前進。
- 遠處或單幀出現的假路牌不會立即觸發動作；必須同時通過信心度、類別框寬及連續影格確認。
- 任一模型或影像處理發生例外時，馬達立即停止並鎖定，避免保留上一次輸出造成失控。
- 兩套模型能在單一相機串流下依序推論，避免重複建立相機造成 `Resource busy`。

實際上的結果

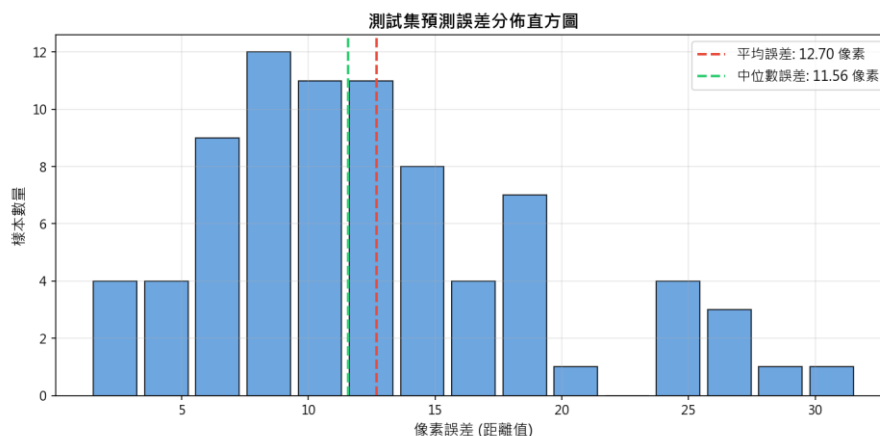
道路模型離線結果

道路資料集共 800 張，影像尺寸均為 224×224 ，檔名座標皆可成功解析且未超出影像範圍。使用固定種子重新抽取 80 張影像進行離線預測檢核，所得結果如下：

指標	結果
平均像素距離誤差	12.702 px
中位數像素距離誤差	11.564 px
標準差	6.674 px
最小誤差	1.377 px
最大誤差	31.653 px
誤差小於 10 px	33 / 80 (41.2%)

此結果用於確認模型輸出與人工座標的相對關係。由於訓練時未保存原始切分索引，因此本次重新抽取的 80 張影像屬於離線抽樣檢核，不宜稱為完全隔離的最終測試集。最大誤差主要出現在急彎、目標點靠近影像邊緣及近端道路變化較大的畫面，因此最新版控制程式加入彎道自動降速，以控制策略補強極端樣本的風險。

圖 4：道路模型離線抽樣的像素誤差分布。



道路模型誤差分布

路牌模型測試結果

YOLOv4-tiny 使用完全隔離的 15 張測試影像，共包含 34 個路牌目標。既有測試報告在信心度門檻 0.30 下得到：

類別	GT	TP	FP	FN	Precision	Recall	F1
stop	7	7	0	0	1.0000	1.0000	1.0000
rail	6	6	2	0	0.7500	1.0000	0.8571
pedestrian	10	9	3	1	0.7500	0.9000	0.8182
blocked	11	11	3	0	0.7857	1.0000	0.8800
平均	34	—	—	—	0.8214	0.9750	0.8888

測試結果顯示模型召回率高，但在較低信心度門檻下仍可能產生重複框或背景虛警。因此最新版 `Final.ipynb` 將實車信心度門檻提高至 0.60，並額外加入框寬與連續兩幀確認，將「模型是否看見路牌」與「是否立即執行車輛動作」分開處理，降低假路牌扣分風險。

最新控制程式驗證結果

最新版 `Final.ipynb` 已完成下列驗證：

- Notebook JSON 結構正確，所有修改過的 Code Cell 均通過 Python 語法解析。
- 道路 `.pth` 權重可成功載入 ResNet-18 並輸出 (1, 2) 張量。
- 道路 ONNX 模型通過 ONNX checker。
- YOLOv4-tiny 設定確認為 416×416 、2 個 `classes=4` 輸出層、2 個 `filters=27` 卷積層，類別順序與控制程式一致。
- 模擬兩個連續 Stop 影格後，馬達停止 2 秒，該次 `callback` 不執行道路推論，避免以舊影像起步。
- 模擬 Pedestrian 漏檢後，減速狀態仍保持 1 秒，再恢復正常速度。
- 模擬 Blocked 後，後續沒有路牌的影格仍保持馬達停止。
- 模擬模型例外後，安全鎖定成立，後續影格不會自行恢復馬達輸出。
- 模擬大轉向量時，`curve_scale` 下降並實際降低基礎速度。

上述結果證明模型檔案、類別映射、部署路徑與最新版控制狀態機在離線環境中一致。TensorRT 引擎仍須在目標 JetBot 上依其 CUDA、TensorRT 與 JetPack 環境首次編譯；最終賽道壓線次數、停止距離及整合推論速度，應以 2026 年 6 月 24 日實體 Demo 的量測紀錄為準。

遇到的問題&問題怎麼解決

遇到的問題	原因分析	解決方法
道路與路牌模型無法分別開啟相機	JetBot CSI 相機及 GStreamer 管線屬於獨佔資源，建立多個 Camera instance 容易產生 Resource busy	單元三只建立一個 416×416 相機實例；同一影格直接供 YOLO 使用，再縮放為 224×224 供道路模型使用
TensorRT 引擎無法在 Windows 預先產生	TensorRT 引擎與 JetBot 的 JetPack、CUDA、TensorRT 版本及硬體綁定	部署包保留 .pth、.onnx、.cfg、.weights，並在 Final.ipynb Cell 2 於目標 JetBot 首次建立兩套引擎
Stop/Rail 停止後可能使用舊影像重新起步	原程式在 time.sleep() 結束後仍繼續處理停止前取得的 img	停止完成並設定 cooldown 後直接 return，等待下一個相機影格重新判斷道路與路牌
遠處假路牌或單幀誤判會觸發動作	原控制只依單一信心度與共用框寬門檻，任何一幀成立就立即執行	使用信心度 0.60、四類獨立框寬、連續兩幀確認及安全優先順序共同決策
不同路牌實際尺寸不同	Rail 路牌形狀與可見寬度不同，若全部使用 50 px 可能過晚觸發	將 Rail 預設門檻設為 30 px，其餘三類預設 50 px，並保留滑桿供現場校正

控制與安全問題

遇到的問題	原因分析	解決方法
Pedestrian 偵測偶爾漏一幀造成速度抖動	每幀重新將速度倍率設為 1.0，單幀漏檢就立刻恢復速度	使用 <code>pedestrian_until</code> 記錄最後確認時間，於 1 秒內持續使用 0.7 倍速度
Blocked 停止後可能因重新觸發程式而移動	僅解除 <code>observer</code> 不能完整表示「永久停止」狀態	使用 <code>blocked_latched</code> 鎖定， <code>callback</code> 開頭先檢查鎖定並持續停止馬達
<code>callback</code> 例外後馬達可能保留上一次輸出	模型、OpenCV 或 Widget 任一例外都可能中斷函式，未執行新的馬達寫入	外層加入 <code>try/except</code> ，例外時立即 <code>robot.stop()</code> 並設定 <code>safety_latched</code> ，必須重新執行啟動單元才能解除
急彎容易因固定速度壓線	道路轉向量增大時仍維持直線巡航速度，車體慣性提高出界風險	依 <code>abs(steering)</code> 計算彎道速度係數，急彎最低降至設定速度的 60%
模型異常輸出可能造成錯誤轉向	TensorRT 或資料處理異常可能產生 NaN、Inf 或超出合理範圍的輸出	檢查輸出長度與有限值，將 X、Y 限制於 <code>[-1, 1]</code> ；不合法時進入安全停止
Stop/Rail 離開路牌前重複觸發	車輛停止後相機仍持續看見同一面路牌	Stop 與 Rail 分別設定 10 秒 <code>cooldown</code> ，使車輛有足夠時間離開路牌區域

3. 本次實驗過程說明與解決方法:

(” 2.實驗過程及結果” 的總結)

本次實驗將道路循跡與交通路牌辨識兩個獨立專案整合為一套 JetBot 自主駕駛系統。道路部分以 800 張人工標定影像訓練 ResNet-18 座標迴歸模型，透過前向距離轉換與 PD 控制器產生左右馬達輸出；路牌部分以 151 張標註影像訓練 YOLOv4-tiny，辨識 stop、rail、pedestrian、blocked 四種類別。兩套模型皆在 JetBot 上轉換為 TensorRT，以符合 Jetson Nano 的即時推論需求。

整合時最重要的設計，是由單一 416×416 相機串流同時供應兩套模型。系統先判斷交通路牌與安全狀態，再執行道路推論與馬達控制，確保 Blocked、Stop、Rail 等安全動作具有優先權。道路控制則使用 $y_forward = \max(1-y, 0.05)$ 避免角度反轉，並加入急彎自動降速，使車輛在轉向量增加時降低前進速度。

為降低期末評分中假路牌誤觸、停止位置超過路牌及三次測試機會不足的風險，最新版 `Final.ipynb` 將路牌動作條件改為信心度、各類框寬與連續影格三重確認；Stop 與 Rail 在停止後等待新影格；Pedestrian 使用 1 秒保持時間；Blocked 與 callback 例外使用鎖定式安全停止。這些修改使控制行為不再只依賴單一影格，並避免程式錯誤後車輛保留舊馬達輸出。

實驗過程中遇到的主要問題包含相機資源衝突、TensorRT 平台相依、舊影格重新起步、假路牌誤觸、Pedestrian 速度抖動、Blocked 停止不夠明確、callback 例外及急彎速度過高。對應解法分別為單一相機影像分流、車端首次編譯、停止後直接返回、連續兩幀與各類門檻、減速保持時間、永久停止鎖定、安全例外鎖定及彎道速度縮放。

目前道路權重、道路 ONNX、YOLOv4-tiny 設定、類別映射、Notebook 語法及控制狀態機均已完成離線驗證，部署包也已整理為可複製至 `trt_yolov4-tiny-master` 的結構。實體 Demo 時應先以 Speed 0.10 至 0.15 低速測試單元一與單元二，再執行單元三整合模式，依實際相機距離微調四類框寬門檻，最後記錄壓線次數、各路牌停止距離與整合推論速度，作為正式驗收結果。